

Texturas em OpenGL Moderno

Texturas em OpenGL Moderno

- Texturas permitem **aplicar imagens sobre superfícies 2D e modelos 3D**, adicionando detalhes visuais **sem aumentar a quantidade de geometria**.
- No OpenGL moderno, as texturas ficam **armazenadas na GPU**, e cada vértice recebe **coordenadas UV** que indicam **qual parte da imagem será utilizada**.
- Durante a renderização, o fragment shader usa essas coordenadas para buscar a cor correspondente na textura.

Nesta aula veremos:

- Texturas 2D e coordenadas UV
- Carregamento de imagens
- Wrapping e filtering
- Mipmaps
- Uso de texturas em shaders

Por que usar texturas?

- Até agora, usamos principalmente cores sólidas, interpolação de cores e iluminação sobre superfícies simples.
- O problema é que **superfícies reais possuem muitos detalhes visuais**, como madeira, tijolos, tecido, pele, metal ou pedra.

Sem texturas:

- Seria necessário **modelar** esses detalhes **diretamente na geometria**, aumentando a quantidade de vértices, faces e processamento.

Com texturas:

- Uma **imagem pode adicionar detalhes visuais à superfície**, permitindo que o objeto continue **geometricamente simples**.

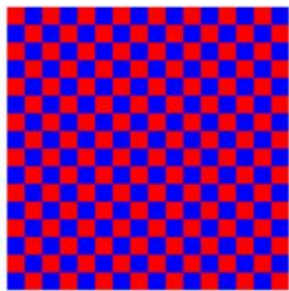


Ideia central do mapeamento de textura

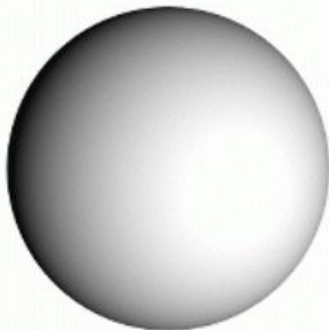
Mapeamento de textura é o processo de **aplicar uma imagem sobre uma superfície**, funcionando como uma “pele” visual do objeto.

A **geometria** define a **forma** da superfície, enquanto a **textura** define seus **detalhes visuais**.

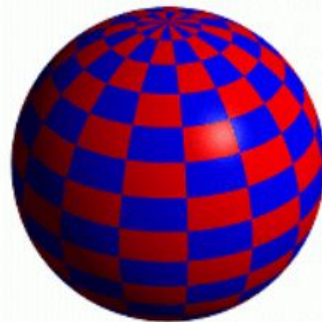
Quando o objeto sofre transformações como translação, rotação ou escala, a **textura acompanha a superfície** automaticamente.



Mapa de textura



Modelo



Modelo texturizado

O que é uma textura?

Uma textura é uma **matriz de dados** armazenada na **memória da GPU**. Em texturas 2D, essa matriz possui **largura, altura e canais de cor**.

A textura também pode armazenar outros tipos de informação além de cores, como **transparência, luminância e outros dados** utilizados pelos shaders.

Cada elemento da textura é chamado de **texel**. Diferente do pixel, que representa um ponto final desenhado na tela, o **texel representa um elemento da própria textura**.



Tipos de textura

Quanto à origem, as texturas podem ser:

- **Bitmap:** imagens formadas por uma matriz de pixels coloridos, como PNG, JPG e BMP.
- **Procedurais:** geradas por algoritmos, como padrões xadrez, ruído, mármore ou madeira sintética.

Quanto à dimensão, as texturas podem ser:

- **1D:** sequência linear de valores, usada por exemplo em gradientes.
- **2D:** imagens com largura e altura, sendo o tipo mais comum em objetos, sprites, interfaces e materiais.
- **3D:** volumes de dados utilizados em casos específicos, como renderização volumétrica e efeitos avançados.

Nesta aula, o foco será em texturas 2D.

Texturas 2D e coordenadas UV

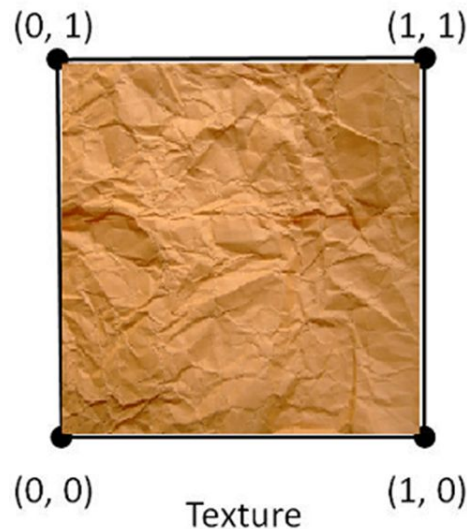
Uma textura 2D é acessada por duas coordenadas:

- **u**: eixo horizontal
- **v**: eixo vertical

Em geral, essas coordenadas variam de 0.0 a 1.0:

- (0.0, 0.0) → canto inferior esquerdo
- (1.0, 0.0) → canto inferior direito
- (1.0, 1.0) → canto superior direito
- (0.0, 1.0) → canto superior esquerdo

O processo de **buscar uma cor da textura** usando coordenadas **UV** é chamado de **sampling**.



Coordenadas de textura por vértice

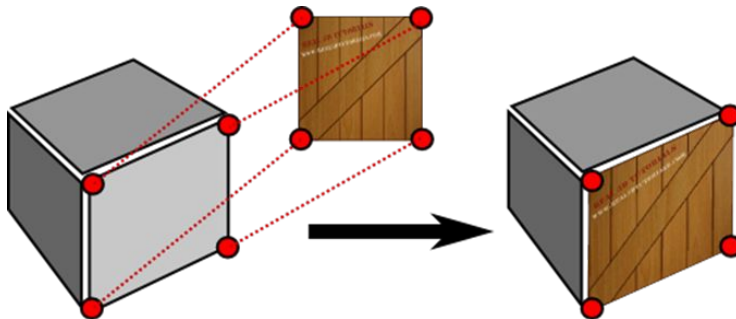
Para aplicar uma textura em uma malha, **cada vértice precisa indicar qual ponto da imagem** corresponde a ele.

Além da **posição** e da **cor**, os **vértices** também podem **armazenar coordenadas de textura**.

A coordenada de textura define a **relação entre a geometria e a imagem**.

Durante a rasterização, essas coordenadas são **interpoladas entre os vértices**, fazendo com que cada **fragmento receba uma coordenada UV intermediária**.

O fragment shader usa essa coordenada para **amostrar a textura**, sem precisar definir manualmente uma coordenada para cada pixel.

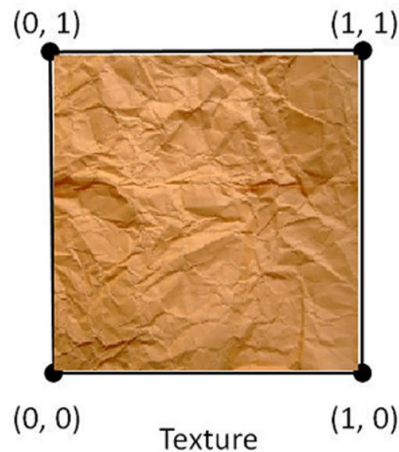


Exemplo: UV em um retângulo

Para texturizar um retângulo inteiro, associamos os quatro cantos do retângulo aos quatro cantos da textura

```
# Formato: x, y, z, u, v
vertices = [
# posição# UV
-0.5, -0.5, 0.0, 0.0, 0.0,# inferior esquerdo
 0.5, -0.5, 0.0, 1.0, 0.0,# inferior direito
 0.5,0.5, 0.0, 1.0, 1.0,# superior direito
-0.5,0.5, 0.0, 0.0, 1.0 # superior esquerdo
]
```

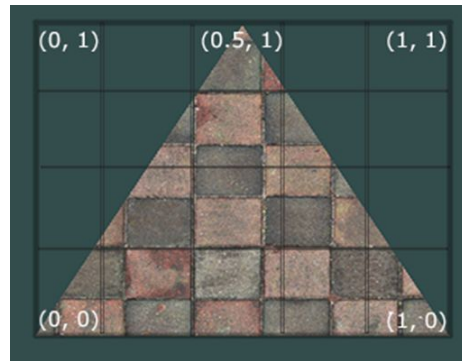
- A **posição** define **onde o vértice aparece** na tela
- A **coordenada UV** define **qual parte da textura** será usada naquele vértice



Exemplo: UV em um triângulo

Em um triângulo, cada vértice também recebe uma coordenada UV

```
# Formato: x, y, z, u, v
vertices = [
# posição# UV
-0.5, -0.5, 0.0, 0.0, 0.0, # vértice inferior esquerdo
 0.5, -0.5, 0.0, 1.0, 0.0, # vértice inferior direito
 0.0, 0.5, 0.0, 0.5, 1.0 # vértice superior central
]
```



- A textura será recortada e interpolada dentro da área do triângulo
- A coordenada 0.5, 1.0 posiciona o topo do triângulo no centro superior da textura

Pipeline de textura em OpenGL moderno

1. Carregar a imagem na CPU

```
image = Image.open('textura.png') #Usando uma biblioteca de imagens como Pillow, por exemplo
```

2. Criar uma textura na GPU

```
texture_id = glGenTextures(1)  
glBindTexture(GL_TEXTURE_2D, texture_id)
```

3. Configurar parâmetros da textura

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT)  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT)  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR)  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR)
```

Pipeline de textura em OpenGL moderno

4. Enviar os dados da imagem para a GPU

```
glTexImage2D(...)
```

5. Usar coordenadas UV no VBO

```
x, y, z, u, v
```

6. Amostrar a textura no fragment shader

```
texture(tex_buffer, texCoord)
```

Criando e vinculando uma textura

Toda textura em OpenGL precisa de um identificador, que representa uma textura armazenada na GPU. Primeiro, criamos o ID da textura:

```
texture_id = glGenTextures(1)
```

Depois, vinculamos esse ID ao alvo GL_TEXTURE_2D:

```
glBindTexture(GL_TEXTURE_2D, texture_id)
```

A partir desse momento, as próximas configurações serão aplicadas à textura vinculada. Em OpenGL, muitas operações funcionam sobre o objeto atualmente associado ao estado ativo.

```
texture_id = glGenTextures(1)
```

```
glBindTexture(GL_TEXTURE_2D, texture_id)
```

Configurando os parâmetros da textura

Depois de vincular a textura, configuramos seu comportamento. Os parâmetros principais são:

- repetição no eixo horizontal da textura
 - `glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT)`
- repetição no eixo vertical da textura
 - `glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT)`
- filtro usado quando a textura é reduzida
 - `glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR)`
- filtro usado quando a textura é ampliada
 - `glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR)`

Configurando os parâmetros da textura

`GL_TEXTURE_WRAP_S` controla o comportamento no eixo u

`GL_TEXTURE_WRAP_T` controla o comportamento no eixo v

`GL_TEXTURE_MIN_FILTER` controla a textura quando ela aparece menor na tela

`GL_TEXTURE_MAG_FILTER` controla a textura quando ela aparece maior na tela

Carregando uma imagem em Python

Antes de enviar a textura para a GPU, precisamos carregar a imagem na CPU

Em Python, podemos usar a biblioteca Pillow:

```
from PIL import Image
```

```
image = Image.open('textura.png')  
image = image.transpose(Image.FLIP_TOP_BOTTOM)  
image = image.convert('RGBA')
```

```
width, height = image.size  
data = image.tobytes()
```

- `Image.open()` carrega o arquivo de imagem
- `FLIP_TOP_BOTTOM` inverte a imagem verticalmente
- `convert('RGBA')` garante quatro canais por texel:
 - vermelho
 - verde
 - azul
 - alpha
- `tobytes()` transforma a imagem em uma sequência de bytes para o OpenGL

Enviando a imagem para a GPU

Com a textura vinculada e a imagem carregada, enviamos os dados para a GPU:

```
glTexImage2D(  
    GL_TEXTURE_2D,  
    0,  
    GL_RGBA,  
    width,  
    height,  
    0,  
    GL_RGBA,  
    GL_UNSIGNED_BYTE,  
    data  
)
```



`GL_TEXTURE_2D`: tipo da textura
`0`: nível base da textura
`GL_RGBA`: formato interno usado pela GPU
`width`: largura da imagem
`height`: altura da imagem
`0`: borda, normalmente zero
`GL_RGBA`: formato dos dados enviados
`GL_UNSIGNED_BYTE`: tipo dos dados de cada canal
`data`: bytes da imagem carregada

Depois do envio, podemos gerar mipmaps:

```
glGenerateMipmap(GL_TEXTURE_2D)
```

Função completa para carregar textura

```
def load_texture(path):
    texture_id = glGenTextures(1)
    glBindTexture(GL_TEXTURE_2D, texture_id)

    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT)
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT)
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR)
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR)

    image = Image.open(path)
    image = image.transpose(Image.FLIP_TOP_BOTTOM)
    image = image.convert('RGBA')

    width, height = image.size
    data = image.tobytes()

    glTexImage2D(
        GL_TEXTURE_2D,
        0,
        GL_RGBA,
        width,
        height,
        0,
        GL_RGBA,
        GL_UNSIGNED_BYTE,
        data
    )
    glGenerateMipmap(GL_TEXTURE_2D)
    return texture_id
```

- A função recebe o caminho da imagem
- Cria e vincula uma textura
- Configura *wrapping* e *filtering*
- Carrega a imagem
- Envia os dados para a GPU
- Retorna o ID da textura

Texture Wrapping

Texture wrapping define o que acontece quando as coordenadas UV ficam fora do intervalo 0.0 a 1.0

Exemplo:

$u = 1.5$

$v = 0.5$

Nesse caso, a coordenada **u** passou do limite direito da textura

O OpenGL precisa decidir como **amostrar a imagem fora da área original**. Os principais modos são:

- `GL_REPEAT`: repete a textura
- `GL_MIRRORED_REPEAT`: repete a textura espelhando a imagem a cada repetição
- `GL_CLAMP_TO_EDGE`: prende a coordenada na borda da textura
- `GL_CLAMP_TO_BORDER`: usa uma cor de borda para coordenadas fora do intervalo

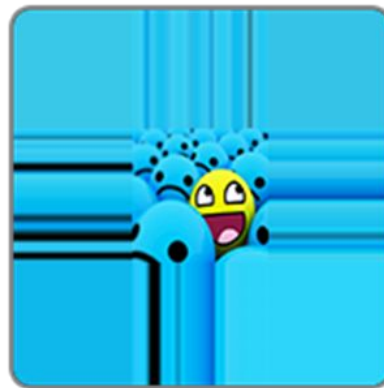
Texture Wrapping



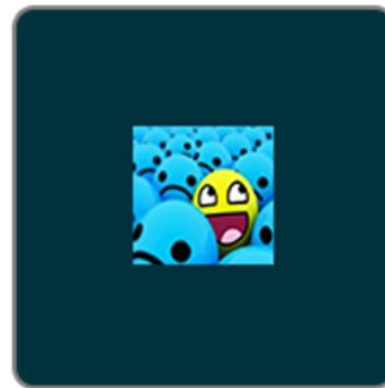
GL_REPEAT



GL_MIRRORED_REPEAT



GL_CLAMP_TO_EDGE



GL_CLAMP_TO_BORDER

Configurando wrapping no código

O wrapping é configurado separadamente para cada eixo da textura.

Eixo horizontal da textura:

```
glTexParameteri(  
    GL_TEXTURE_2D,  
    GL_TEXTURE_WRAP_S,  
    GL_MIRRORED_REPEAT  
)
```

Eixo vertical da textura:

```
glTexParameteri(  
    GL_TEXTURE_2D,  
    GL_TEXTURE_WRAP_T,  
    GL_MIRRORED_REPEAT  
)
```

Configurando wrapping no código

- `GL_TEXTURE_WRAP_S` controla o eixo u
- `GL_TEXTURE_WRAP_T` controla o eixo v

Para usar `GL_CLAMP_TO_BORDER`, definimos uma cor de borda:

```
border_color = [1.0, 1.0, 0.0, 1.0]
glTexParameterfv(
    GL_TEXTURE_2D,
    GL_TEXTURE_BORDER_COLOR,
    border_color
)
```

Essa cor será usada quando a coordenada UV estiver fora do intervalo 0.0 a 1.0

Texture Filtering

Texture filtering define como o OpenGL calcula a cor de um fragmento a partir dos texels da textura.

O problema aparece quando a textura não é exibida em seu tamanho original.

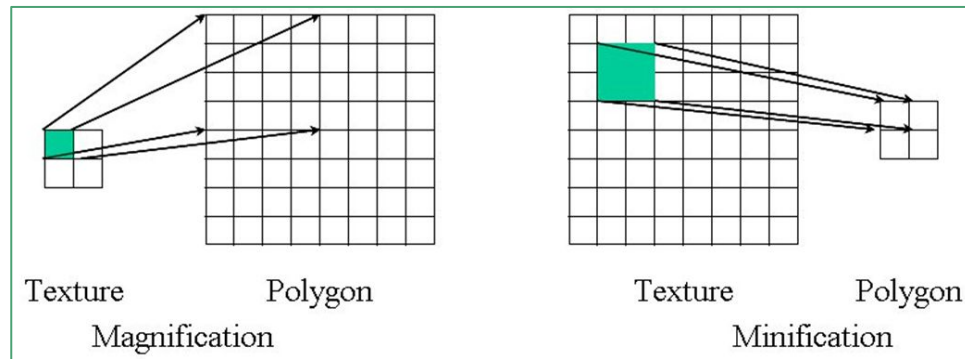
Existem dois casos principais:

- **magnificação**

- a textura aparece maior na tela
- poucos texels precisam preencher muitos pixels

- **minificação**

- a textura aparece menor na tela
- muitos texels precisam caber em poucos pixels



O filtro define como a cor final será **escolhida** ou **interpolada**

GL_NEAREST

GL_NEAREST usa o texel mais próximo da coordenada UV amostrada.

Características:

- cálculo simples e geralmente mais rápido
- não suaviza a imagem
- gera aparência pixelada

É útil quando queremos preservar pixels bem marcados, como em:

- pixel art
- jogos retrô
- texturas que não devem ser borradas

```
glTexParameteri(  
    GL_TEXTURE_2D,  
    GL_TEXTURE_MAG_FILTER,  
    GL_NEAREST  
)
```

```
glTexParameteri(  
    GL_TEXTURE_2D,  
    GL_TEXTURE_MIN_FILTER,  
    GL_NEAREST  
)
```


GL_LINEAR

GL_LINEAR interpola os texels vizinhos da coordenada UV amostrada.

Características:

- suaviza a textura
- reduz aparência pixelada
- pode deixar a imagem levemente borrada
- costuma ser melhor para texturas realistas

Exemplo comum:

- madeira
- parede
- tecido
- pele
- objetos 3D com texturas fotográficas

```
glTexParameteri(  
    GL_TEXTURE_2D,  
    GL_TEXTURE_MAG_FILTER,  
    GL_LINEAR  
)
```

```
glTexParameteri(  
    GL_TEXTURE_2D,  
    GL_TEXTURE_MIN_FILTER,  
    GL_LINEAR  
)
```

Comparação: GL_NEAREST vs GL_LINEAR

GL_NEAREST: escolhe o texel mais próximo, mantendo bordas duras e um resultado mais pixelado.

GL_LINEAR: interpola texels vizinhos, suavizando a imagem e produzindo um resultado mais borrado.

A escolha do filtro depende do estilo visual:

- Pixel art → GL_NEAREST
- Textura realista → GL_LINEAR



GL_NEAREST



GL_LINEAR

Mipmaps

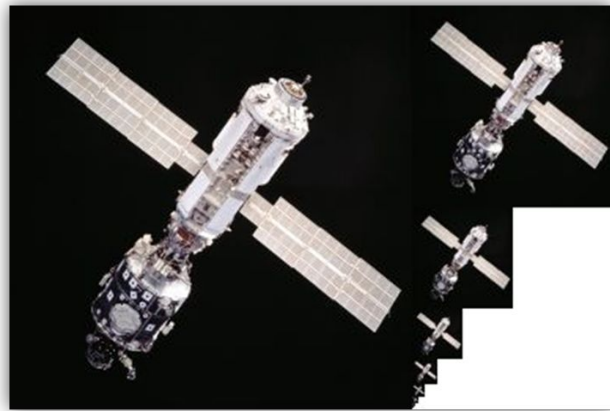
Mipmaps são **versões menores da mesma textura**, criadas pelo OpenGL reduzindo a imagem pela metade sucessivamente. Por exemplo:

- 1024 x 1024
- 512 x 512
- 256 x 256
- 128 x 128
- 64 x 64
- e assim por diante.

Eles são usados quando a textura aparece pequena na tela. Sem mipmaps, texturas distantes podem causar ruído visual e serrilhado. Com mipmaps, o OpenGL escolhe o nível mais adequado ao tamanho visível.

Para gerar mipmaps, usamos:

```
glGenerateMipmap(GL_TEXTURE_2D)
```



Filtros com mipmaps

Quando usamos mipmaps, o **filtro de minificação** pode escolher entre diferentes níveis da textura

Filtros comuns:

- GL_NEAREST_MIPMAP_NEAREST
- GL_LINEAR_MIPMAP_NEAREST
- GL_NEAREST_MIPMAP_LINEAR
- GL_LINEAR_MIPMAP_LINEAR

O filtro mais usado para boa qualidade visual é:

```
glTexParameteri(  
    GL_TEXTURE_2D,  
    GL_TEXTURE_MIN_FILTER,  
    GL_LINEAR_MIPMAP_LINEAR  
)
```

Filtros com mipmaps

Para **magnificação**, mipmap não é usado. Por isso, o filtro de ampliação normalmente continua sendo:

```
glTexParameteri(  
    GL_TEXTURE_2D,  
    GL_TEXTURE_MAG_FILTER,  
    GL_LINEAR  
)
```

A combinação típica fica assim:

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR)  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR)
```

Vértices com coordenadas UV

Para usar texturas, cada vértice precisa carregar coordenadas UV. Neste exemplo, cada vértice terá 8 valores:

- x, y, z, r, g, b, u, v

A estrutura dos vértices fica assim:

```
vertices = np.array([
# posição# cor # UV
-0.5, -0.5, 0.0, 1.0, 0.0, 0.0,0.0, 0.0,
 0.5, -0.5, 0.0, 0.0, 1.0, 0.0,1.0, 0.0,
 0.5,0.5, 0.0, 0.0, 0.0, 1.0,1.0, 1.0,
-0.5,0.5, 0.0, 1.0, 1.0, 1.0,0.0, 1.0
], dtype=np.float32)
```

- A **posição** define a **geometria**
- A **cor** pode ser usada para **modulação**
- A **coordenada UV** define a parte da **textura associada ao vértice**

Índices do retângulo

Um retângulo pode ser desenhado com dois triângulos. Para evitar repetir vértices, usamos um EBO

```
indices = np.array([  
0, 1, 2,  
2, 3, 0  
, dtype=np.uint32)
```

Primeiro triângulo:

- 0, 1, 2

Segundo triângulo:

- 2, 3, 0

Os índices indicam quais vértices do array serão usados em cada triângulo.

Configurando VAO, VBO e EBO

```
VAO = glGenVertexArrays(1)
VBO = glGenBuffers(1)
EBO = glGenBuffers(1)

glBindVertexArray(VAO)

glBindBuffer(GL_ARRAY_BUFFER, VBO)
glBufferData(
    GL_ARRAY_BUFFER,
    vertices.nbytes,
    vertices,
    GL_STATIC_DRAW
)

glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO)
glBufferData(
    GL_ELEMENT_ARRAY_BUFFER,
    indices.nbytes,
    indices,
    GL_STATIC_DRAW
)
```

VAO armazena a configuração dos atributos dos vértices

VBO armazena os dados dos vértices

EBO armazena os índices dos triângulos

Configurando os atributos dos vértices

Cada vértice possui 8 valores do tipo float: x, y, z, r, g, b, u, v

Portanto, o stride é: `stride = 8 vertices.itemsize`

Atributo de posição:

```
glVertexAttribPointer(0,3,GL_FLOAT,GL_FALSE,stride, ctypes.c_void_p(0))  
glEnableVertexAttribArray(0)
```

Atributo de cor:

```
glVertexAttribPointer(1,3,GL_FLOAT,GL_FALSE,stride,ctypes.c_void_p(3 vertices.itemsize))  
glEnableVertexAttribArray(1)
```

Atributo de coordenada UV:

```
glVertexAttribPointer(2,2,GL_FLOAT,GL_FALSE,stride,ctypes.c_void_p(6 vertices.itemsize))  
glEnableVertexAttribArray(2)
```

Vertex shader com coordenadas UV

```
#version 330 core
```

```
layout (location = 0) in vec3 position;  
layout (location = 1) in vec3 color;  
layout (location = 2) in vec2 tex_coord;
```

```
out vec3 vertexColor;  
out vec2 texCoord;
```

```
uniform mat4 model;  
uniform mat4 projection;
```

```
void main()  
{  
    gl_Position = projection model vec4(position, 1.0);  
  
    vertexColor = color;  
    texCoord = tex_coord;  
}
```

`location = 0` recebe a posição

`location = 1` recebe a cor

`location = 2` recebe a coordenada UV

`texCoord` será interpolada automaticamente durante a rasterização

O fragment shader receberá a coordenada UV interpolada

Fragment shader com sampler2D

```
#version 330 core
```

```
in vec3 vertexColor;
```

```
in vec2 texCoord;
```

```
out vec4 fragColor;
```

```
uniform sampler2D tex_buffer;
```

```
void main()
```

```
{
```

```
fragColor = texture(tex_buffer, texCoord);
```

```
}
```

- sampler2D representa uma textura 2D no shader
- texCoord é a coordenada UV recebida do vertex shader
- texture(tex_buffer, texCoord) busca a cor da textura naquela coordenada
- O valor retornado é um vec4 com canais RGBA

Enviando o sampler para o shader

O `sampler2D` do shader precisa saber qual unidade de textura será usada. Primeiro, ativamos uma unidade de textura

```
glActiveTexture(GL_TEXTURE0)
```

Depois, vinculamos a textura nessa unidade

```
glBindTexture(GL_TEXTURE_2D, texture_id)
```

Em seguida, informamos ao shader que `tex_buffer` usará a unidade 0

```
shader.use()
```

```
location = glGetUniformLocation(shader.program, 'tex_buffer')
```

```
glUniform1i(location, 0)
```

`GL_TEXTURE0` corresponde ao valor 0, por isso usamos `glUniform1i(location, 0)`

Desenhando com textura

Na hora de desenhar, precisamos garantir que:

- o shader correto está ativo
- a textura correta está vinculada
- o VAO correto está vinculado

`shader.use()`

```
glActiveTexture(GL_TEXTURE0)
glBindTexture(GL_TEXTURE_2D, texture_id)
```

```
glBindVertexArray(VAO)
```

```
glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_INT, None)
```

- `glDrawElements` usa os índices armazenados no EBO. O fragment shader usa a textura vinculada para calcular a cor final dos fragmentos.

Misturando cor do vértice com textura

Podemos usar apenas a textura:

```
fragColor = texture(tex_buffer, texCoord);
```

Também podemos combinar a textura com a cor do vértice:

```
fragColor = texture(tex_buffer, texCoord) * vec4(vertexColor, 1.0);
```

Nesse caso:

- a textura fornece os detalhes visuais
- a cor do vértice altera a tonalidade da textura
- o resultado final é uma modulação entre cor e textura

Isso é útil para preparar o caminho para materiais e iluminação

Múltiplas texturas

Um shader pode acessar mais de uma textura. Para isso, usamos diferentes unidades de textura

```
glActiveTexture(GL_TEXTURE0)  
glBindTexture(GL_TEXTURE_2D, texture_1)
```

```
glActiveTexture(GL_TEXTURE1)  
glBindTexture(GL_TEXTURE_2D, texture_2)
```

Cada textura é vinculada a uma unidade diferente

- texture_1 -> GL_TEXTURE0
- texture_2 -> GL_TEXTURE1

No shader, usamos dois samplers:

```
uniform sampler2D texture1;  
uniform sampler2D texture2;
```

No Python, associamos cada sampler à unidade correta:

```
glUniform1i(glGetUniformLocation(shader.program, 'texture1'), 0)  
glUniform1i(glGetUniformLocation(shader.program, 'texture2'), 1)
```

Misturando duas texturas no fragment shader

```
#version 330 core
in vec2 texCoord;
out vec4 fragColor;

uniform sampler2D texture1;
uniform sampler2D texture2;

void main()
{
    vec4 color1 = texture(texture1, texCoord);
    vec4 color2 = texture(texture2, texCoord);

    fragColor = mix(color1, color2, 0.5);
}
```

- texture1 busca a cor da primeira textura
- texture2 busca a cor da segunda textura
- mix(color1, color2, 0.5) mistura as duas cores em proporções iguais

Alterando o terceiro parâmetro, mudamos o peso da mistura:

```
mix(color1, color2, 0.2)
mix(color1, color2, 0.8)
```

Esse recurso permite combinar imagens, máscaras, detalhes e efeitos visuais em uma mesma superfície.